CMPSC 463: Design and Analysis of Algorithms (Spring 2026)

Project-2:
SafeRoute

By:
Zainab Afridi & Yuliana Gryb

Instructor
Janghoon Yang

Submitted On: 4/26/2026

TABLE OF CONTENTS

1. DESCRIPTION OF THE PROJECT

SafeRoute is a browser-based navigation tool for conflict and disaster zones. People in a crisis often need to reach a hospital, food source, water point, or shelter quickly, with limited information about which roads are passable. SafeRoute helps by finding the shortest path through a road network modeled as a weighted graph.

The app models the crisis area as 18 locations (nodes) connected by 60 roads (edges). Each edge has a cost representing travel difficulty. The app answers two questions:

- Single destination: what is the cheapest path from my location to the nearest resource of a given type?
- Multi-destination: if I need to visit several stops, in what order should I visit them?

The whole app runs in one HTML file with no backend or internet connection needed after loading. That was a deliberate choice since network access is often the first thing lost in a crisis.

2. SIGNIFICANCE OF THE PROJECT

Routing under uncertainty is a real problem in humanitarian response. Aid workers and civilians have to make fast decisions about where to go with incomplete information. The graph algorithms we studied this semester are a natural fit for this kind of problem, and SafeRoute makes that connection tangible.

A few things make this project meaningful:

- The graph model is grounded in reality. Edge weights can represent distance, road damage, or checkpoint delays.
- BFS runs before Dijkstra to check if a destination is even reachable. In conflict zones, some areas really do get cut off.
- The greedy multi-stop approach matches how people think in the field: go to the nearest thing first, then figure out the next step.
- The app works offline, which makes it actually usable in low-connectivity situations.
- Edge costs are labeled directly on the map, so users can verify the output themselves without needing to understand the code.

What makes it somewhat novel is the combination: a lightweight single-file app that uses BFS and Dijkstra together for humanitarian routing, with edge weights visible on screen so the algorithm's reasoning is transparent.

This project demonstrates how classical algorithms can be adapted to real-world, high-stakes environments where decisions must be made quickly and with limited information.

3. CODE STRUCTURE

The app is one HTML file, but the code splits into three logical layers:

Data Layer

- NODES[]: 18 location objects with id, label, type, and x/y position
- EDGES[]: 60 entries in [nodeA, nodeB, cost] format
- adjList[]: adjacency list built from EDGES on startup, used by BFS and Dijkstra

Algorithm Layer

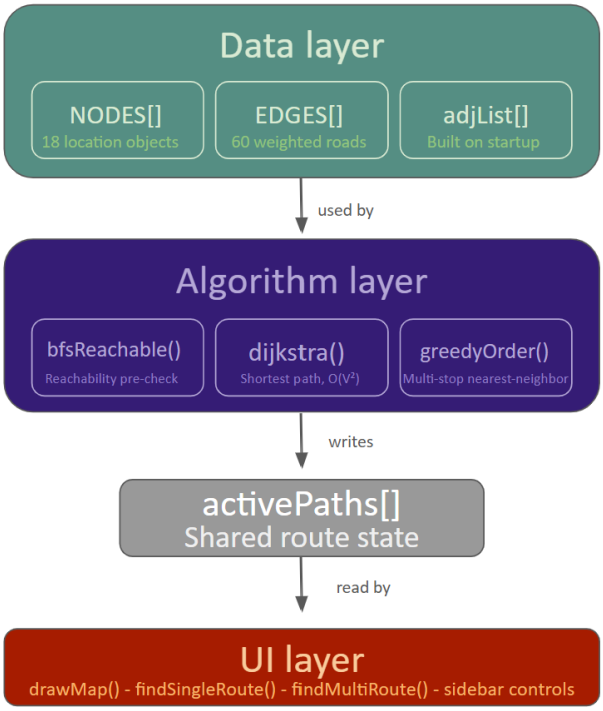
- bfsReachable(start, end): checks if a path exists using BFS
- dijkstra(start, end): finds the shortest path using dist[], prev[], and backtracking
- greedyOrder(start, destIds): picks the nearest unvisited stop each time, calls Dijkstra repeatedly
- getEdgeCost(a, b): returns the cost between two adjacent nodes

UI Layer

- drawMap(): renders nodes, edges, route highlights, and cost labels on a canvas
- findSingleRoute(): runs BFS then Dijkstra for the single-destination flow
- findMultiRoute(): runs BFS then greedyOrder for multi-stop planning
- setMode(), buildMultiCheckboxes(), updateStats(), populateStartSel(): sidebar controls

Data flows one way: user input triggers an algorithm function, which updates activePaths[], which drawMap() reads to redraw the canvas.

Figure 1: Code Architecture Diagram



4. DESCRIPTION OF ALGORITHMS

BFS (Breadth-First Search)
BFS runs before Dijkstra as a reachability check. It uses a FIFO queue, expanding nodes level by level. If it reaches the destination, a path exists. If the queue empties, we skip Dijkstra and show a warning. This matters because in a conflict zone some nodes really can become unreachable.
Time complexity: O(V + E)

Dijkstra's Algorithm
Dijkstra finds the shortest path in a graph with non-negative edge weights. Our implementation:

- Sets dist[start] = 0, all others = Infinity
- Each step picks the unvisited node with the smallest known distance
- Relaxes each neighbor: if going through the current node is cheaper, update that neighbor's distance and record it in prev[]
- Backtracks through prev[] at the end to reconstruct the path

Time complexity: O(V^2) with an array-based queue. Fast enough for 18 nodes and easier to read than a heap-based version.

Greedy Nearest-Neighbor (Multi-Stop)
Multi-stop routing is a variant of the Travelling Salesman Problem (TSP), which is NP-hard. Instead of checking all orderings, we use a greedy approach: always go to the cheapest unvisited stop next. It is not globally optimal but is fast and practical.
Time complexity: O(k * V^2) where k is the number of stops (capped at 4).

5. VERIFICATION OF ALGORITHMS

Dijkstra: Toy Example

Graph: A-B (4), A-C (2), B-D (3), C-B (1), C-D (7), D-E (2). Find the shortest path from A to E.

- Start: dist[A]=0, all others = Infinity
- Visit A: dist[B]=4, dist[C]=2
- Visit C (cost 2): dist[B] = min(4, 3) = 3, dist[D] = 9
- Visit B (cost 3): dist[D] = min(9, 6) = 6
- Visit D (cost 6): dist[E] = 8
- Visit E (cost 8). Done.

Result: A → C → B → D → E, total cost 8. The direct route A → B → D → E costs 9. Dijkstra correctly picks the cheaper path.

BFS: Reachability Check

Same graph, checking if A can reach E:

- Start with queue [A]. Enqueue neighbors B and C.
- Dequeue B, enqueue D.
- Dequeue C, no new neighbors.
- Dequeue D, enqueue E.
- Dequeue E. Destination found. Return true.

If the D-E edge did not exist, the queue would empty and BFS returns false. Dijkstra never runs.

Greedy: Multi-Stop Example

Start at A, need to visit B, C, and D:

- From A: nearest is C (cost 2). Go to C.
- From C: nearest is B (cost 1). Go to B.
- From B: only D left (cost 3). Go to D.

Total: 6. The ordering A → B → C → D would cost 12. The greedy approach finds a much better route.

6. EXECUTION RESULTS AND ANALYSIS

Single-Destination Mode

The user picks a start location and destination type. BFS filters out unreachable nodes first, then Dijkstra finds the cheapest path. The route highlights in green on the map with edge costs labeled, and the sidebar shows each step.

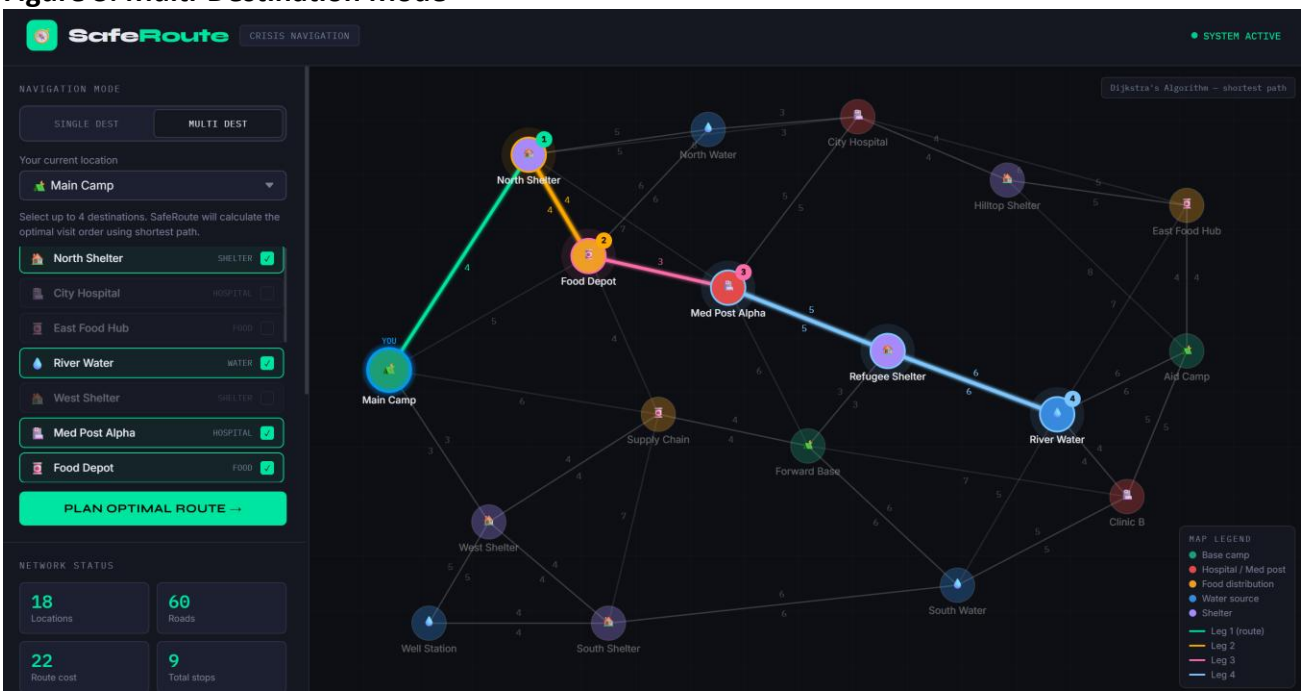
Example: Main Camp to the nearest hospital. Dijkstra finds Main Camp → Food Depot → Med Post Alpha at cost 8 (edges 5+3). There is a longer path to City Hospital, but this one is cheaper.

Figure 2: Single-Destination Mode

Multi-Destination Mode

The user checks up to 4 destinations. The greedy heuristic plans the visit order. Each leg gets a different color on the map (green, orange, pink, blue) with a numbered badge on each destination node. The sidebar shows cost per leg and the total.

Figure 3: Multi-Destination Mode



Observations

- Single-destination always returns the optimal path. Dijkstra guarantees this for non-negative edge weights.
- Multi-destination is not globally optimal, but beats random orderings in practice.
- BFS catches unreachable destinations right away and shows a warning instead of silently failing.
- All computation finishes under a millisecond. The graph is small enough that performance is not a concern.

These results show that Dijkstra's algorithm consistently produces optimal routes in weighted graphs, even when multiple alternatives exist. The greedy multi-destination approach sacrifices optimality but significantly improves speed, which is critical in time-sensitive crisis scenarios. This highlights an important tradeoff between optimal solutions and practical efficiency.

7. CONCLUSIONS

SafeRoute showed us that the algorithms from this course have real, practical uses. BFS, Dijkstra, and greedy heuristics are exactly what you would use in a humanitarian routing problem, and building a working tool around them made the tradeoffs much more concrete.

What We Accomplished

- Modeled a crisis road network as a weighted undirected graph
- Used BFS to catch unreachable destinations before running Dijkstra
- Used Dijkstra to guarantee optimal paths in single-destination mode
- Used greedy nearest-neighbor for fast multi-stop routing
- Made results verifiable by displaying edge costs directly on the map

Known Issues

- The graph is static. Roads cannot be blocked or updated in real time.
- Greedy multi-stop is not globally optimal. With 4 stops there are 24 possible orderings and we only check one.
- Edge costs are made-up numbers. A real version would need actual distance or travel-time data.

Course Connections

This project used graph representations with adjacency lists, BFS, Dijkstra's algorithm, greedy heuristics, and time complexity analysis. Choosing array-based Dijkstra (O(V^2)) over a heap version, and greedy over exact TSP, were tradeoff decisions based on what we covered in class.

8. AI USAGE

AI tools were used as assistants during development, not as a replacement for our own work. We used AI (Claude and ChatGPT) to help clarify algorithm concepts such as Dijkstra’s algorithm and BFS, debug portions of JavaScript code, and improve the structure and wording of the project report. AI was also used to suggest UI improvements and help organize the code into logical layers.

AI tools were used only as support for understanding concepts, debugging, and improving clarity, while all final coding, algorithm design, and testing decisions were made independently by us.